

Pandas:

- Pandas is a powerful Python library used for data manipulation and analysis.
- Import pandas as pd (short form)

```
>> import pandas as pd
```

- It provides flexible and efficient data structures for handling structured data.

The two primary data structures in Pandas are:

(i) Series:

A one-dimensional labeled array capable of holding data of a single type.

(ii) DataFrame:

A two-dimensional labeled data structure with columns of potentially different types (numeric, string, Boolean, etc).

Feature	Series	DataFrame
Dimension	1D	2D
Example	One column	Table

Examples:

(i) A Pandas series:

0	4
1	6
2	-5
3	3

Where, 0, 1, 2 etc are index and the second column is the series of data.

If the series is named as "ser",

```
>> ser = pd.Series([4, 6, -5, 3]) # creates the above Series.
```

We can consider

We create a series with preferred index,

```
>> ser = pd.Series([4, 6, -5, 3], index = ["b", "c", "a", "d"])
```

Creates a series,

b	4
c	6
a	-5
d	3

(ii) A DataFrame:

	country	year	population
0	India	2002	2.5
1	China	2005	2.8
2	Brazil	2004	1.1
3	Japan	2006	0.5

Where 0, 1, 2, 3 are index, three columns with headers "country", "year" and "population".

We can create the above DataFrame,

```
Data = {"country": ["India", "China", "Brazil", "Japan"],
        "year": [2002, 2005, 2004, 2006],
        "population": [2.5, 2.8, 1.1, 0.5]}

dF = pd.DataFrame(Data)
```

Will create the dataframe shown above. A DataFrame can be created in multiple ways and the above illustration is one among those.

```
Let newdata = {"country": ["India", "China", "Brazil", "Japan", "Germany",
                           "Italy", "Russia", "Spain", "Peru", "Kenya"],
               "year": [2002, 2005, 2004, 2006, 2001, 2003, 2002, 2005, 2004,
                        2006 ],
               "population": [2.5, 2.8, 1.1, 0.5, 1.2, 2.1, 1.6, 1.9, 2.0, 1.8]}
```

If we specify a sequence of columns, the DataFrame's columns will be arranged in that order:

```
>> dF = pd.DataFrame(newdata, columns = ["year", "country",
"population"])
```

Modifies the DataFrame dF as,

	year	country	population
0	2002	India	2.5
1	2005	China	2.8

2	2004	Brazil	1.1
3	2006	Japan	0.5
4	2001	Germany	1.2
5	2003	Italy	2.1
6	2002	Russia	1.6
7	2005	Spain	1.9
8	2004	Peru	2.0
9	2006	Kenya	1.8

For large DataFrames, the head method returns only the first five rows.

```
>> print(dF.head())
```

	year	country	population
0	2002	India	2.5
1	2005	China	2.8
2	2004	Brazil	1.1
3	2006	Japan	0.5
4	2001	Germany	1.2

Similarly, tail returns the last five rows.

```
>> print(dF.tail())
```

	year	country	population
--	------	---------	------------

5	2003	Italy	2.1
6	2002	Russia	1.6
7	2005	Spain	1.9
8	2004	Peru	2.0
9	2006	Kenya	1.8

Important note : head(m) returns first m rows and tail(n) returns last n rows.

dF.country or dF["country"] returns only the column "country"

```
>> print(dF.country)
```

	country
0	India
1	China
2	Brazil
3	Japan
4	Germany
5	Italy
6	Rusia
7	Spain
8	Peru
9	Kenya

dtype: object

If we choose to display more than one columns and all rows of a DataFrame,

Then,

```
>> print(dF[["country", "population"]])
```

	country	population
0	India	2.5
1	China	2.8
2	Brazil	1.1
3	Japan	0.5
4	Germany	1.2
5	Italy	2.1
6	Rusia	1.6
7	Spain	1.9
8	Peru	2.0
9	Kenya	1.8

Important note: use double square bracket for multiple columns.

`df.dtypes` will return columns and its data types

```
>> print(df.dtypes)
```

```
country    object
```

```
year       int64
```

```
population float64
```

dtype: object

`df.describe()` gives a quick statistical summary of the data.

```
>> print(df.describe())
```

	year	population
count	10.00000	10.000000
mean	2003.80000	1.750000
std	1.75119	0.681909
min	2001.00000	0.500000
25%	2002.25000	1.300000
50%	2004.00000	1.850000
75%	2005.00000	2.075000
max	2006.00000	2.800000

`sort_index()` can be used to sort the dataframe based on the index. In the DataFrame `df` considered above, this will have no effect because the index is a serial number which is already in order.

`sort_values()` can be used to sort the dataframe based on any column.

```
>> df.sort_values(by="population", ascending = True, inplace =
False)
```

- "by" provides us option to choose the column to be used for sorting.
- `sort_values()` can have arguments like "ascending" and "inplace"

Argument	Purpose	Default
ascending	Order of sorting	True
inplace	Modify original or not	False

- "inplace" modifies the original DataFrame if True

	country	year	population
3	Japan	2006	0.5
2	Brazil	2004	1.1
4	Germany	2001	1.2
6	Rusia	2002	1.6
9	Kenya	2006	1.8
7	Spain	2005	1.9
8	Peru	2004	2.0
5	Italy	2003	2.1
0	India	2002	2.5
1	China	2005	2.8

`dF.columns` provides the column names of the DataFrame.

```
>> print(dF.columns)
```

```
Index(['country', 'year', 'population'], dtype='object')
```

If we create a DataFrame using a dictionary and add the column which does not exist in the dictionary e.g "GDP" in the above example. Then, it creates a column with null values.

```
>> dF = pd.DataFrame(newdata, columns = ["year ", "country",
    "population", "GDP"])
```

```
>> print(dF)
```


	country	year	population	GDP
0	India	2002	2.5	NaN
1	China	2005	2.8	NaN
2	Brazil	2004	1.1	NaN
3	Japan	2006	0.5	NaN
4	Germany	2001	1.2	NaN
5	Italy	2003	2.1	NaN
6	Rusia	2002	1.6	NaN
7	Spain	2005	1.9	NaN
8	Peru	2004	2.0	NaN
9	Kenya	2006	1.8	NaN

For deleting a column, we can use del,

```
>> del dF["GDP"]
```

	country	year	population
0	India	2002	2.5
1	China	2005	2.8
2	Brazil	2004	1.1
3	Japan	2006	0.5
4	Germany	2001	1.2
5	Italy	2003	2.1
6	Rusia	2002	1.6
7	Spain	2005	1.9
8	Peru	2004	2.0
9	Kenya	2006	1.8

isna():

In large datasets, it is possible to have no values in some cells of a table,

We can find all those null values using isna().

```
Let ndata = {"country": ["India", "China", "Brazil", "Japan", "Germany",  
                        "Italy", "Russia", "Spain", "Peru", "Kenya"],  
            "year": [2002, 2005, 2004, 2006, 2001, 2003, 2002, 2005, 2004,  
                    2006 ],  
            "population": [2.5, 2.8, None, 0.5, 1.2, 2.1, 1.6, 1.9, 2.0, None]}
```

```
>> dF = pd.DataFrame(ndata)  
  
>> print(dF.isna())
```

	country	year	population
0	False	False	False
1	False	False	False
2	False	False	True
3	False	False	False
4	False	False	False
5	False	False	False
6	False	False	False
7	False	False	False
8	False	False	False
9	False	False	True

The above code returns a Boolean array where True indicates missing data (None/NaN) and False indicates valid data.

```
>> nF = dF[dF.isna().any(axis=1)]
```

	country	year	population
2	Brazil	2004	NaN
9	Kenya	2006	NaN

`any(axis=1)` returns True for rows that contain at least one missing value..
These rows are then selected from the DataFrame.

fillna():

`fillna()` can be used to replace missing values with a specified value.

```
>> print(dF.fillna("NotAvailable"))
```

	country	year	population
0	India	2002	2.5
1	China	2005	2.8
2	Brazil	2004	NotAvailable
3	Japan	2006	0.5
4	Germany	2001	1.2
5	Italy	2003	2.1
6	Russia	2002	1.6
7	Spain	2005	1.9
8	Peru	2004	2.0
9	Kenya	2006	NotAvailable

`fillna()` replaces the None values with "NotAvailable". We can replace with any values or a string.

Slicing a DataFrame:

There are two types of slicing. One is label based slicing(.loc) and the other is position based slicing(iloc).

Pandas uses the label in the index when slicing the DataFrame with "loc".

```
>> print(dF.loc[2:5, :]) # prints the rows whose index label is between
```

2 to 5 and all the columns of dF.

	country	year	population
2	Brazil	2004	1.1
3	Japan	2006	0.5
4	Germany	2001	1.2
5	Italy	2003	2.1

```
>> dF.set_index("year") # uses the year as index
```

```
>> print(dF)
```

	country	population
year		
2002	India	2.5
2005	China	2.8
2004	Brazil	1.1
2006	Japan	0.5
2001	Germany	1.2
2003	Italy	2.1
2002	Rusia	1.6
2005	Spain	1.9
2004	Peru	2.0
2006	Kenya	1.8

If we use "loc" to slice the DataFrame, we have to use years not numerals because we have changed the index to be used.

```
>> dF.sort_index(inplace = True) # because labels must be sorted for  
                                     slicing  
  
>> print(dF.loc[2003:2006, :])
```

	country	population
year		
2003	Italy	2.1
2004	Brazil	1.1
2004	Peru	2.0
2005	Spain	1.9
2005	China	2.8
2006	Japan	0.5
2006	Kenya	1.8

Using loc, we select rows whose index labels range from 2003 to 2006, along with all columns.

Using iloc, a DataFrame is sliced by numerical positions (0, 1, 2, ...), ignoring the actual index labels.

```
>> print(dF.iloc[0:4, :]) # slices first 4 rows
```

Provides output as,

	country	year	population
0	India	2002	2.5
1	China	2005	2.8
2	Brazil	2004	1.1
3	Japan	2006	0.5

```
>> print(dF.iloc[4:8, 0:2]) #slice row 4 to 7 and column 0 and 1
```

Produces,

	country	year
4	Germany	2001
5	Italy	2003
6	Rusia	2002
7	Spain	2005

- "loc" includes the end point, while "iloc" excludes the end point
- "loc" slicing is label based while "iloc" is based on integer position
- For both rows and columns, slicing conditions applicable
- If we know the labels, we can use "loc" for slicing otherwise we can use "iloc"

Filtering data:

In Pandas, filtering is used to retrieve a subset of data by applying conditions to columns. This helps in focusing only on relevant data for analysis.

```
data = {'Name': ['Raghav', 'Sourabh', 'Veer', 'Iyer', 'Gupta', 'Kumar', 'Sharma'],  
        'Age': [25, 32, 45, 19, 27, 18, 29],
```

'Score': [85, 90, 78, 95, 35, 48, 71]}

```
>> dF = pd.DataFrame(data)
```

We can filter the data using condition as follows,

```
>> dF_filtered = dF[dF["Age"] > 30]
```

```
>> print(dF_filtered)
```

	Name	Age	Score
1	Sourabh	32	90
2	Veer	45	78

```
>> new_DF = dF[(dF["Age"] < 30) | (dF["Score"] > 50)]
```

```
>> print(new_DF)
```

	Name	Age	Score
0	Raghav	25	85
3	Iyer	19	95
6	Sharma	29	71

```
>> new_DF = dF[(dF["Age"] < 30) & (dF["Score"] > 70)]
```

```
>> print(new_DF)
```

	Name	Age	Score
0	Raghav	25	85
3	Iyer	19	95
4	Gupta	27	35
5	Kumar	18	48
6	Sharma	29	71

```
>> new_DF = dF.loc[dF['Age'] > 30, ['Name', 'Score']]
```

Using the above, we can select the columns that are interested.

	Name	Score
1	Sourabh	90
2	Veer	78

```
>> new_DF = dF[~(dF['Age'] > 26)] # Filters Age < 26
```

	Name	Age	Score
0	Raghav	25	85
3	Iyer	19	95
5	Kumar	18	48

‘~’ is used for logical "NOT" operations.

If we have the values of the items to filter, we can use `isin()`.

```
>> new_DF = dF[dF['Age'].isin([18, 19])]
```


	Name	Age	Score
3	Iyer	19	95
5	Kumar	18	48